

Logistic Regression for Wisconsin Breast Cancer Data Set

October 7, 2026

0.0.1 Logistic Regression for Wisconsin Breast Cancer Data

This section will look at applying logistic regression to the Wisconsin Breast Cancer Dataset. We begin by using filter feature selection to fit an unregularized model and then look at using a regularized model.

```
[143]: #We begin by importing all the libraries we will be using in this section
import pandas as pd
import random
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import l1_min_c
import matplotlib.pyplot as plt
from sklearn.base import clone
from sklearn.metrics import log_loss
from sklearn.metrics import ConfusionMatrixDisplay
from sklearn.model_selection import FixedThresholdClassifier
from sklearn.frozen import FrozenEstimator

from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import f_classif
```

```
[144]: #we load the data into a dataframe
df1 = pd.read_csv("data.csv")
df1.head()
```

```
[144]:
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	\
0	842302	M	17.99	10.38	122.80	1001.0	
1	842517	M	20.57	17.77	132.90	1326.0	
2	84300903	M	19.69	21.25	130.00	1203.0	
3	84348301	M	11.42	20.38	77.58	386.1	
4	84358402	M	20.29	14.34	135.10	1297.0	

	smoothness_mean	compactness_mean	concavity_mean	concave	points_mean	\
0	0.11840	0.27760	0.3001		0.14710	

1	0.08474	0.07864	0.0869	0.07017
2	0.10960	0.15990	0.1974	0.12790
3	0.14250	0.28390	0.2414	0.10520
4	0.10030	0.13280	0.1980	0.10430

	texture_worst	perimeter_worst	area_worst	smoothness_worst	\
0	17.33	184.60	2019.0	0.1622	
1	23.41	158.80	1956.0	0.1238	
2	25.53	152.50	1709.0	0.1444	
3	26.50	98.87	567.7	0.2098	
4	16.67	152.20	1575.0	0.1374	

	compactness_worst	concavity_worst	concave points_worst	symmetry_worst	\
0	0.6656	0.7119	0.2654	0.4601	
1	0.1866	0.2416	0.1860	0.2750	
2	0.4245	0.4504	0.2430	0.3613	
3	0.8663	0.6869	0.2575	0.6638	
4	0.2050	0.4000	0.1625	0.2364	

	fractal_dimension_worst	Unnamed: 32
0	0.11890	NaN
1	0.08902	NaN
2	0.08758	NaN
3	0.17300	NaN
4	0.07678	NaN

[5 rows x 33 columns]

```
[145]: df = df1.iloc[:, :-1].dropna() #data cleaning; getting rid of na
df.head()
```

```
[145]:
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	\
0	842302	M	17.99	10.38	122.80	1001.0	
1	842517	M	20.57	17.77	132.90	1326.0	
2	84300903	M	19.69	21.25	130.00	1203.0	
3	84348301	M	11.42	20.38	77.58	386.1	
4	84358402	M	20.29	14.34	135.10	1297.0	

	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	\
0	0.11840	0.27760	0.3001	0.14710	
1	0.08474	0.07864	0.0869	0.07017	
2	0.10960	0.15990	0.1974	0.12790	
3	0.14250	0.28390	0.2414	0.10520	
4	0.10030	0.13280	0.1980	0.10430	

	radius_worst	texture_worst	perimeter_worst	area_worst	\
0	25.38	17.33	184.60	2019.0	

1	...	24.99	23.41	158.80	1956.0
2	...	23.57	25.53	152.50	1709.0
3	...	14.91	26.50	98.87	567.7
4	...	22.54	16.67	152.20	1575.0

	smoothness_worst	compactness_worst	concavity_worst	concave points_worst	\
0	0.1622	0.6656	0.7119		0.2654
1	0.1238	0.1866	0.2416		0.1860
2	0.1444	0.4245	0.4504		0.2430
3	0.2098	0.8663	0.6869		0.2575
4	0.1374	0.2050	0.4000		0.1625

	symmetry_worst	fractal_dimension_worst
0	0.4601	0.11890
1	0.2750	0.08902
2	0.3613	0.08758
3	0.6638	0.17300
4	0.2364	0.07678

[5 rows x 32 columns]

```
[146]: #Cross Validation
#splitting into training and test data
X = df.drop(columns="diagnosis")
y = df["diagnosis"]
x_train_full = X.sample(frac=0.8, random_state=1)
x_train = x_train_full.sample(frac=0.875, random_state=2)
x_val = x_train_full.drop(index=x_train.index)
x_test = X.drop(index=x_train_full.index)

y_train_full = (y.loc[x_train_full.index]=="M").astype(int)
y_train = (y.loc[x_train.index]=="M").astype(int)
y_val = (y.loc[x_val.index]=="M").astype(int)
y_test = (y.loc[x_test.index]=="M").astype(int)
```

0.0.2 Feature Selection

At this stage we would like to choose how many and which features to use. In this section we use a function from SciKit Learn [1] called SelectKBest, where best is judged based on their `f_classif` function, to choose the most informative features. This is an example of a filtering method, which means the features are chosen based on their qualities before a model is fitted and not based on model performance, which could require computing a different model for each possible combination of features. The choice once K is known does not depend on the model, but the choice of K itself might. This will be our hyperparameter which we will choose at the validation stage.

```
[148]: selectors = {
        k: SelectKBest(f_classif, k=k).fit(x_train, y_train)
```

```

    for k in [5, 10, "all"]
}

splits = {"train": x_train, "val": x_val, "test": x_test}

selected = {
    k: {
        split: X.loc[:, selector.get_support()]
        for split, X in splits.items()
    }
    for k, selector in selectors.items()
}

```

The function `SelectKBest` using a scoring function (in this case `f_classif`) to assign a score to each column (feature) and then returns the columns with the highest scores. The scoring function, `f_classif`, measures the relationship between the feature and the diagnosis, `y` within the training data. In this case, `f_classif` measures the ANOVA f-statistic which measures if each feature's mean values differ across classes, relative to the variation within classes. The f-statistic for feature `j` is calculated for a binary case as follows:

$$F_j = \frac{\sum_{c=0}^1 n_c (\bar{x}_{cj} - \bar{x}_j)^2}{\frac{1}{n-2} \sum_{c=0}^1 \sum_{i:y_i=c} (x_{ij} - \bar{x}_{cj})^2}$$

Where `c` represents the classes and `n` is the number of observations, and n_c is the number of observations in class `c`.

This statistic helps us find useful features because features which are more concentrated within a class than they are across the whole dataset are likely to contain more information about `y`: a higher f-statistic means the feature is more spread out across both classes than it is within one class by a lot. Having computed our best features, we use the Logistic Regression function from SciKit Learn to perform logistic regression on our data. We have chosen 3 candidates for `k`: 5, 10 and all, as there are only 31 in total. We read about the loss function that is minimised by the Logistic Regression function in [6] after reading about logistic regression as a whole in [3].

```

[149]: clf1 = make_pipeline(
    StandardScaler(),
    LogisticRegression(
        C=np.inf,
        solver="lbfgs",
        max_iter=5000
    ),
)

results1={}
models1={}
for k in selected:
    model1 = clone(clf1)
    model1.fit(selected[k]["train"], y_train)
    p_fit = model1.predict_proba(selected[k]["train"])

```

```

p_val = model1.predict_proba(selected[k]["val"])
models1[k]=model1
results1[k] = {
    "training_log_loss": log_loss(
        y_train, p_fit, labels=model1.classes_
    ),
    "validation_log_loss": log_loss(
        y_val, p_val, labels=model1.classes_
    )
}
pd.DataFrame(results1)

```

```

[149]:
           5           10          all
training_log_loss    0.114168  0.103179  0.000243
validation_log_loss  0.108217  0.083654  0.564694

```

The training and validation loss functions clearly show us that the model with 5 features has reasonable performance, but that 10 features improves on it for both training and validation loss. We can see that using all features clearly overfits, giving us a very small training loss value and a large validation loss value. Based on this we choose 10 features for our final model and asses it's accuracy on the test data.

```

[150]: x_train_full_10= x_train_full[selected[10]["train"].columns]

final_model1 = clone(models1[10])
final_model1.fit(x_train_full_10, y_train_full)
print(f"Test accuracy: {final_model1.score(selected[10]["test"], y_test):.2%}")

```

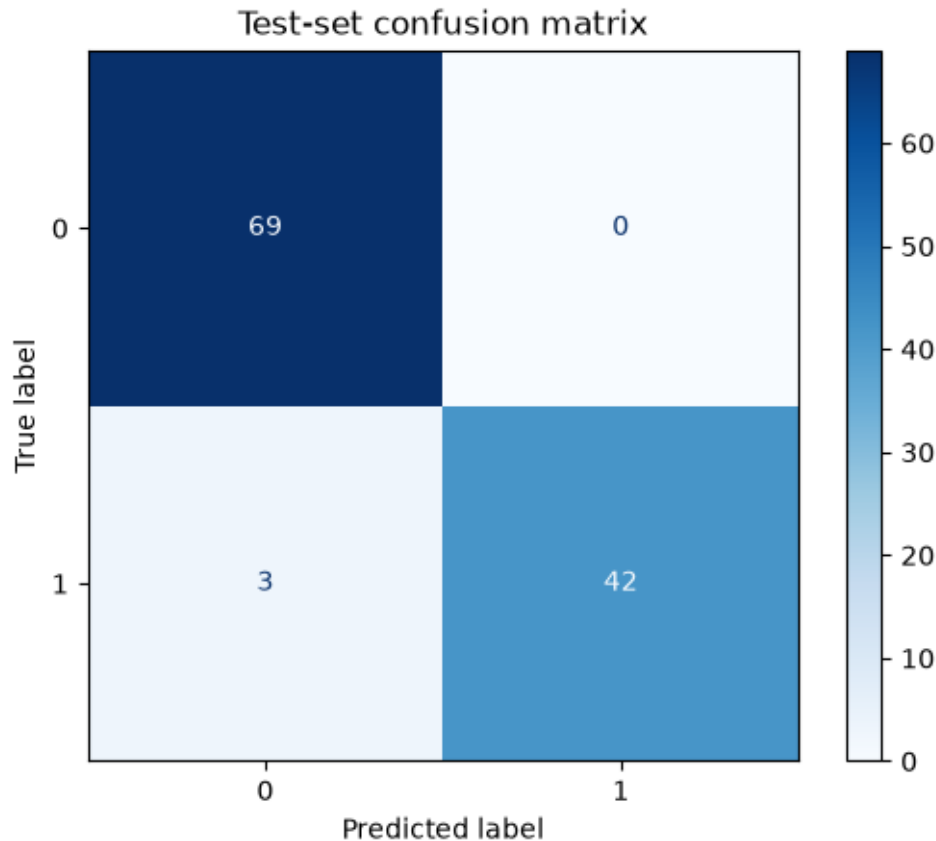
Test accuracy: 97.37%

The test accuracy is extremely high for such a simple model, so this suggests that for this dataset this was a successful approach. Better cross validation, such as leave one out cross validation, would involve retraining the model on different splits of training, validation, and testing data, but that is beyond the scope of this review. To further investigate the performance of the model, we look at the confusion matrix.

```

[160]: ConfusionMatrixDisplay.from_estimator(
    final_model1,
    selected[10]["test"],
    y_test,
    cmap="Blues",
    values_format="d"
)
plt.title("Test-set confusion matrix")
plt.show()

```



We can see that this method achieved 3 false positives and 0 false positives. In the context of cancer screening, this is not ideal: we would prefer to have a false positive and intervene when not necessary, or do further testing, then to give a false negative and leave someone with an untreated malignant tumour. One approach to addressing this within logistic regression is discussed later.

0.0.3 Regularized Logistic Regression

In regularized logistic regression, we modify the loss function to penalize complexity and avoid overfitting. In this report we have used regularization as an alternative to feature selection but they could be used in conjunction. In this section we have used SciKit Learn's Logistic Regression Function again but with a different solver so that we could adjust the regularization parameter, c . We begin by initializing a few candidate values of c . These values are on a logarithmic scale so that they scale with the loss function, which is also logarithmic. Unlike the standard λ parameter, the c parameter control regularization in the opposite direction: a smaller c represents a higher penalty and a more regularized model. The code below has been adapted from the code in [7] to store the models and to simplify the candidates for c .

```
[163]: cs = [0.01, 0.1, 1, 10, 100]
```

```
[152]: clf = make_pipeline(
        StandardScaler(),
        LogisticRegression(
            l1_ratio=1,
            solver="liblinear",
            tol=1e-6,
            max_iter=int(1e6),
            warm_start=True,
            fit_intercept=False,
        ),
    )

models = {}
results = []

for c in cs:
    model = clone(clf)
    model.set_params(logisticregression__C=c)
    model.fit(x_train, y_train)

    p_fit = model.predict_proba(x_train)
    p_val = model.predict_proba(x_val)

    models[c] = model

    results.append({
        "C": c,
        "training_log_loss": log_loss(
            y_train, p_fit, labels=model.classes_
        ),
        "validation_log_loss": log_loss(
            y_val, p_val, labels=model.classes_
        )
    })

results = pd.DataFrame(results)
results
```

```
[152]:
```

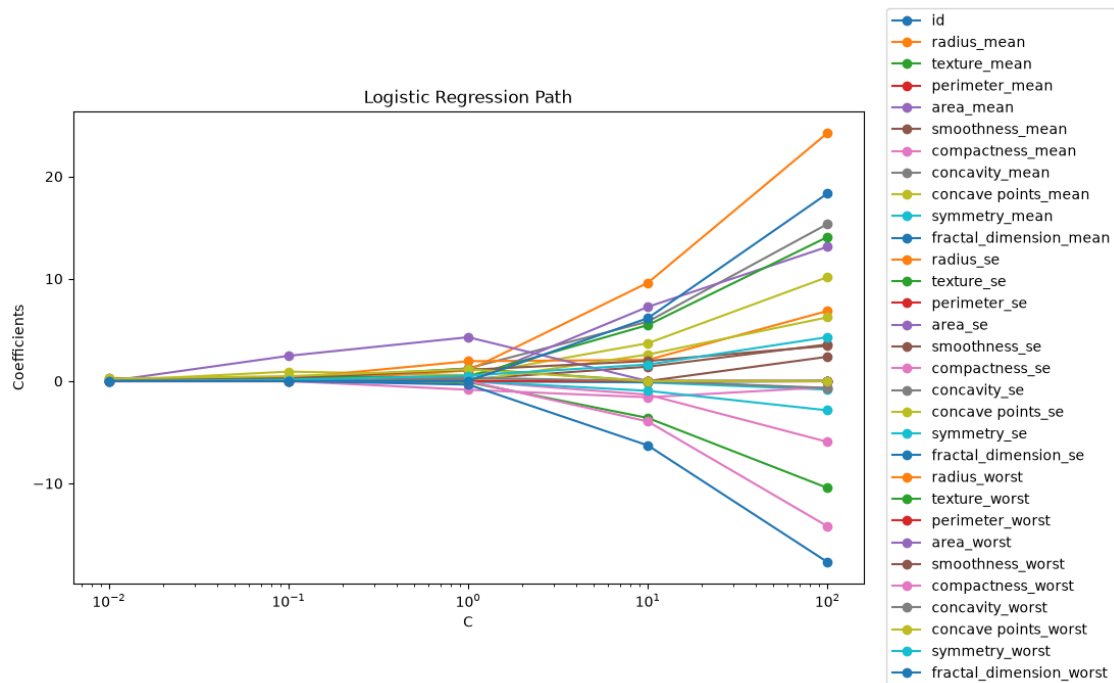
	C	training_log_loss	validation_log_loss
0	0.01	0.513906	0.501732
1	0.10	0.129768	0.123065
2	1.00	0.046423	0.064725
3	10.00	0.011852	0.077597
4	100.00	0.001263	0.157690

We can see that a very small value of c punishes large coefficients too much and majorly underfits, while a value of 100 clearly overfits. Based on this data the best choice for c is 1.

The following code has been adapted from [7] and shows how the coefficients of each feature varied depending on c . We can see that a small c punishes large coefficients so severely that all coefficients are 0. In effect this means the probability of a one of the classifiers will be 1 regardless of the features. If we interpreted the log loss as the estimated probability of misclassification, we'd expect this to be around 0.5, which is what we observe.

```
[164]: plt.figure(figsize=(10, 6))
for i in range(coefs_.shape[1]):
    plt.semilogx(cs, coefs_[i], marker="o", label=x_train.columns[i])

ymin, ymax = plt.ylim()
plt.xlabel("C")
plt.ylabel("Coefficients")
plt.title("Logistic Regression Path")
plt.legend(loc="center left", bbox_to_anchor=(1.02, 0.5))
plt.axis("tight")
plt.show()
```



```
[181]: coefficients = pd.Series(
    models[10]["logisticregression"].coef_[0],
    index=x_train.columns,
    name="coefficient"
)

ranked = coefficients.loc[
```

```

    coefficients.abs().sort_values(ascending=False).index
]

print(ranked.head(5))
print(", ".join(selected[5]["train"].columns))

```

```

radius_worst          9.605956
area_se               7.254029
fractal_dimension_se  -6.287725
fractal_dimension_worst  6.149757
concavity_mean        5.813331
Name: coefficient, dtype: float64
perimeter_mean, concave points_mean, radius_worst, perimeter_worst, concave
points_worst

```

We can see that when c is larger, the coefficients grow more for features which are more useful, but this is not consistent with the most useful features measured using the f -statistic. The best features according to the f -statistic are: `perimeter_mean`, `concave points_mean`, `radius_worst`, `perimeter_worst`, `concave points_worst`. The features with the highest coefficients are: `radius_worst`, `areas_se`, `fractaldimension_se`, `fractaldimension_worst`, `concavity_mean`. It is interesting to see that two models which use completely different features can achieve similar performance, and the performance can be high. We think this is a reflection of the richness of the data.

```

[179]: final_model = clone(models[1])
final_model.fit(x_train_full, y_train_full)
print(f"Test accuracy: {final_model.score(x_test, y_test):.2%}")

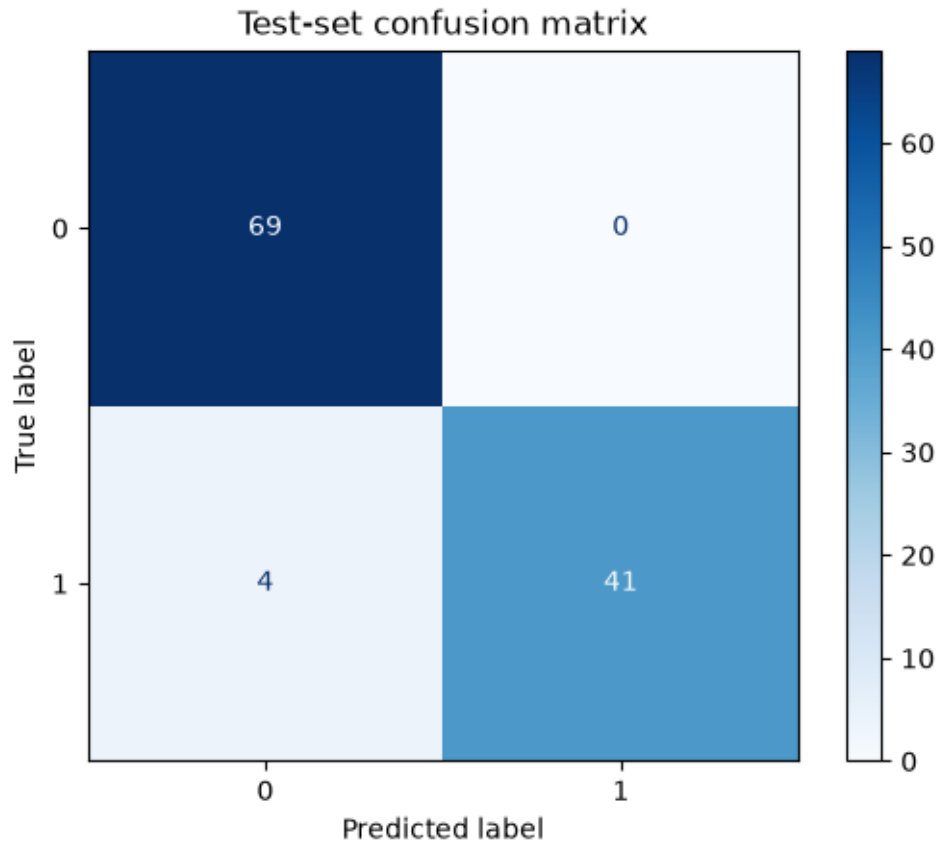
```

Test accuracy: 96.49%

```

[180]: ConfusionMatrixDisplay.from_estimator(
    final_model,
    x_test,
    y_test,
    cmap="Blues",
    values_format="d"
)
plt.title("Test-set confusion matrix")
plt.show()

```

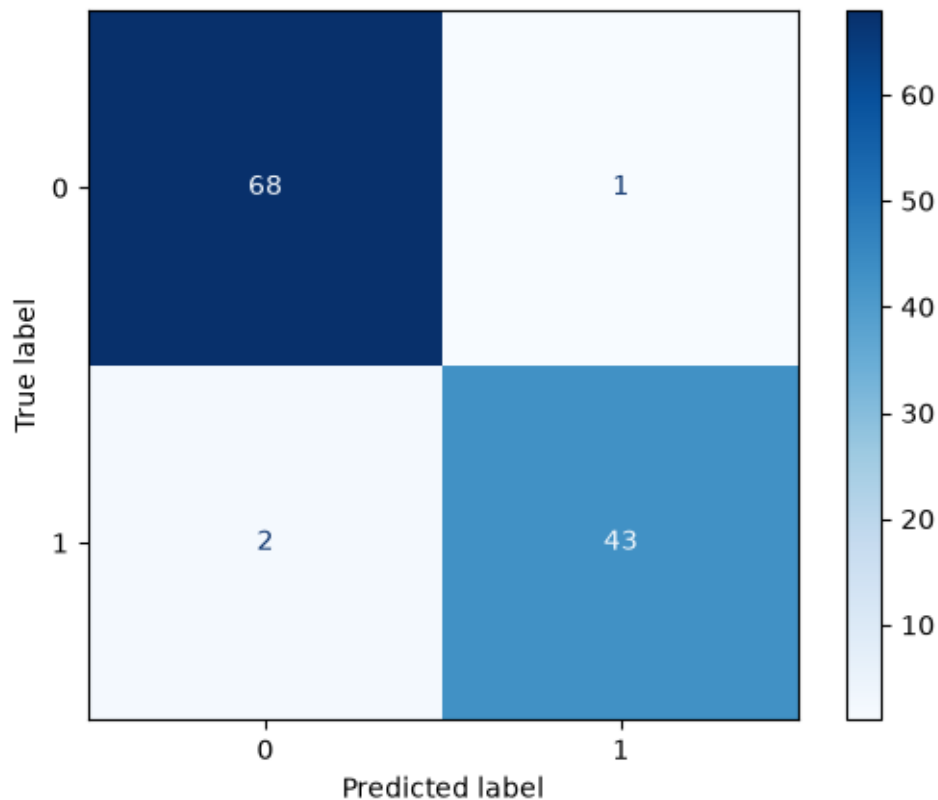


As previously discussed, the false positives are not ideal for the context that this is a medical classification problem, so below we try to address this by introducing a lower threshold to classify a tumour as malignant.

```
[156]: threshold_model = FixedThresholdClassifier(  
    estimator=FrozenEstimator(final_model),  
    threshold=0.3,  
    pos_label=1,  
    response_method="predict_proba"  
)  
  
ConfusionMatrixDisplay.from_estimator(  
    threshold_model,  
    x_test,  
    y_test,  
    cmap="Blues",  
    values_format="d"  
)
```

```
print(f"Test accuracy: {threshold_model.score(x_test, y_test):.2%}")
```

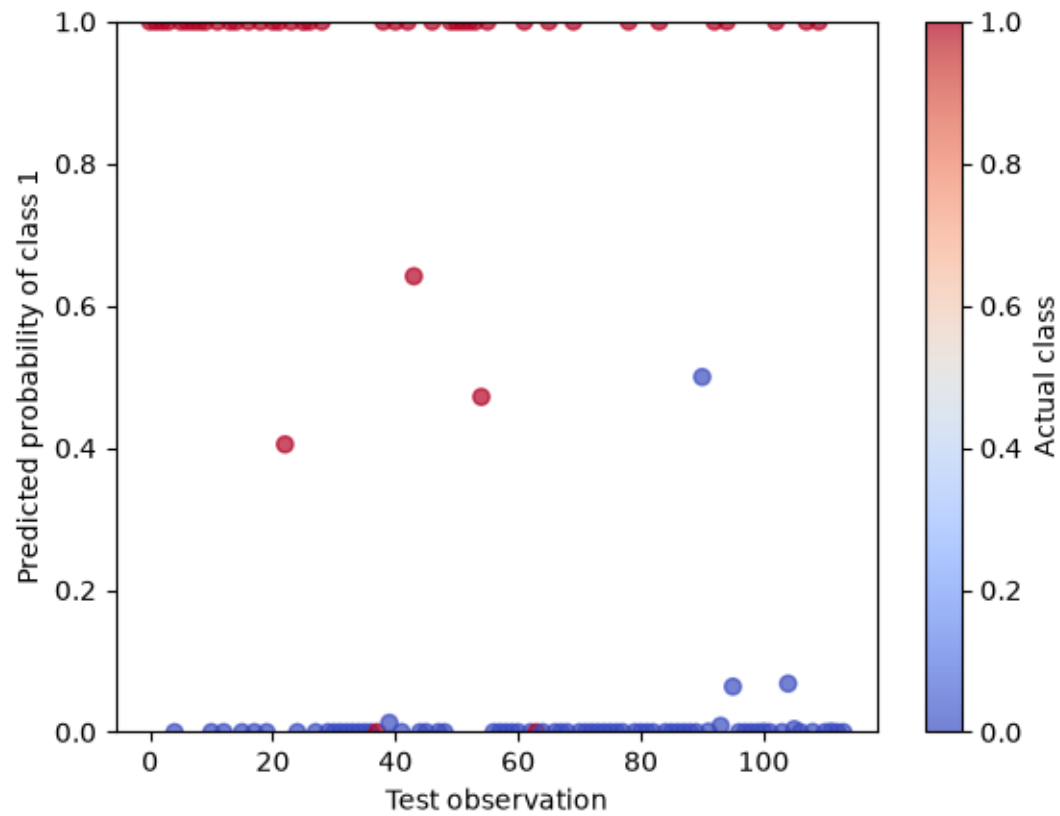
Test accuracy: 97.37%



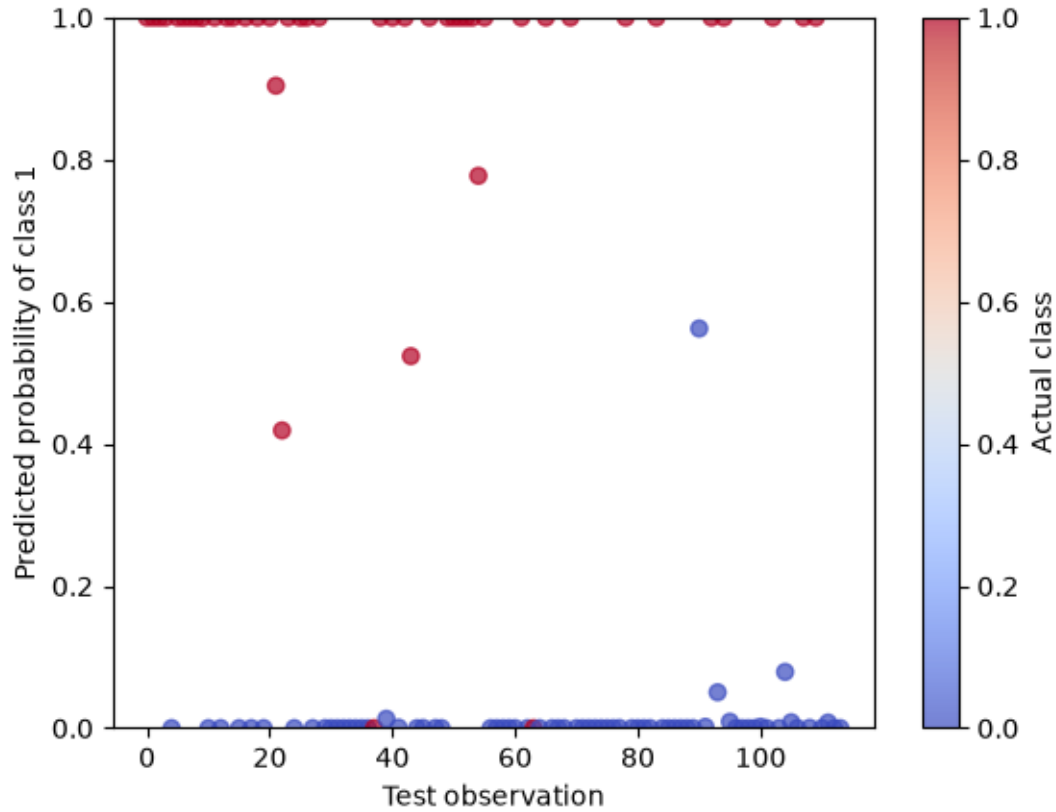
For this particular dataset, this improved the accuracy, although this is now overfitting because we are adjusting the model to our testing data, but the principle remains that we could lower the threshold for classifying a malignant tumour to avoid false negatives at the cost of introducing more false positives. While adjusting the threshold we noticed it was quite hard to make the model change its mind, so we decided to plot the predictions of the models with a different c parameter to see why the model was not sensitive to being adjusted.

```
[182]: p = models[10].predict_proba(x_test)[: , 1]

plt.scatter(range(len(p)), p, c=y_test, cmap="coolwarm", alpha=0.7)
plt.xlabel("Test observation")
plt.ylabel("Predicted probability of class 1")
plt.colorbar(label="Actual class")
plt.ylim(0, 1)
plt.show()
```



```
[158]: p=final_model.predict_proba(x_test)[: ,1]
plt.scatter(range(len(p)), p, c=y_test, cmap="coolwarm", alpha=0.7)
plt.xlabel("Test observation")
plt.ylabel("Predicted probability of class 1")
plt.colorbar(label="Actual class")
plt.ylim(0, 1)
plt.show()
```



Based on these plots we concluded that a larger c parameter, by allowing the coefficients to grow, was increasing how certain the model was about its prediction, making it less sensitive to a change in threshold. Infact for both models we plotted there were only a few points that the model was uncertain about, meaning threshold changing may not be a good method for logistic regression false negative avoidance.

We conclude based on this analysis that logistic regression works well with the Wisconsin Breast Cancer Dataset as we achieved high levels of accuracy, and that feature filtering had comparable performance to regularized regression for this data set, making it a powerful tool for logistic regression.

[1] https://scikit-learn.org/stable/modules/feature_selection.html

[2] <https://blog.minitab.com/en/blog/adventures-in-statistics-2/understanding-analysis-of-variance-anova-and-the-f-test>

[3] [https://www.datacamp.com/tutorial/understanding-logistic-regression-python?utm_cid=23340058065&utm_aid=192632748929&utm_campaign=230119_1-ps-dscia_{dsa-tofu}python_2-b2c_3-europe_4-prc_5-na_6-na_7-le_8-pdsh-go_9-nb-e_10-na_11-na&utm_loc=9045630-&utm_mtd=-c&utm_kw=&utm_source=google&utm_medium=paid_search&utm_con-dscia_{europe-en}dsa_{tofu}tutorial~python&gad_source=1&gad_campaignid=23340058065&gbraid=0AAAAADQ9WsGZ](https://www.datacamp.com/tutorial/understanding-logistic-regression-python?utm_cid=23340058065&utm_aid=192632748929&utm_campaign=230119_1-ps-dscia_dsa-tofu_python_2-b2c_3-europe_4-prc_5-na_6-na_7-le_8-pdsh-go_9-nb-e_10-na_11-na&utm_loc=9045630-&utm_mtd=-c&utm_kw=&utm_source=google&utm_medium=paid_search&utm_con-dscia_europe-en_dsa-tofu_tutorial~python&gad_source=1&gad_campaignid=23340058065&gbraid=0AAAAADQ9WsGZ)

[4] <https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>

- [5] https://cs229.stanford.edu/main_notes.pdf
- [6] https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
- [7] https://scikit-learn.org/stable/auto_examples/linear_model/plot_logistic_path.html